

Single Agent Systems & Agent Pipelines

1. Explain the concept of a stateful directed graph in agent pipelines. How does it differ from a simple linear pipeline?

Answer: A stateful directed graph is a workflow where each step can store and use information from previous steps. The workflow is made up of nodes and edges that define how data moves through the system. It can support branching, looping, and decision-making. This makes it more flexible and intelligent. In contrast, a linear pipeline follows a fixed sequence of steps from start to finish without remembering previous states or changing its path.

2. Describe the role of nodes and edges in an agent workflow. Give an example of each.

Answer: Nodes are the individual tasks or actions performed in an agent workflow. They can represent operations such as processing a query, calling a tool, or generating a response. Edges are the connections between nodes that determine how information flows. For example, a "Calculator Tool" can be a node, while the path connecting query analysis to the calculator is an edge. Together, nodes and edges define the complete workflow.

3. What is conditional routing in an agent system? Design a simple rule-based routing logic for three different query types.

Answer: Conditional routing is the process of directing a query to different tools or actions based on its intent. It helps the agent choose the most appropriate response method. For example, if a query contains the word "calculate", it is sent to the Calculator Tool. If it contains "keywords", it is sent to the Keyword Extraction Tool. All other queries can be handled by a General Response module.

4. Why are cycles (loops) important in agent pipelines? Provide a use case where a retry loop is necessary.

Answer: Cycles or loops allow an agent to repeat a process until a desired result is achieved. They are useful when tasks may fail or require multiple attempts. For example, if an API request fails due to a temporary network issue, the agent can retry the request instead of immediately returning an error. This improves reliability and robustness. Without loops, the workflow would stop after a single failure.

5. Explain how a single-agent system can simulate multi-agent behavior internally.

Answer: A single-agent system can simulate multiple agents by dividing its work into different roles. It may first analyze the query, then decide which tool to use, and finally generate a response. Each role behaves like a separate agent even though they are all executed by one system. This approach reduces complexity while maintaining flexibility. As a result, a single agent can perform tasks similar to a multi-agent architecture.

6. What are JSON schema tools? How do they help in structuring tool inputs and outputs?

Answer: JSON schema tools define a standard structure for data exchanged between agents and tools. They specify required fields, data types, and expected formats. This ensures that inputs are valid before processing

begins. It also makes outputs consistent and easier to interpret. Using JSON schemas reduces errors and improves communication between different components of an agent system.

7. Compare sequential tool calls and parallel tool calls. When would you prefer one over the other?

Sequential tool calls are executed one after another, where each step depends on the result of the previous step. Parallel tool calls execute multiple independent tasks at the same time. Sequential execution is preferred when tasks have dependencies. Parallel execution is useful when tasks are unrelated and can be completed simultaneously. Using parallel calls can significantly reduce response time and improve efficiency.

8. How would you implement error handling in a tool-using agent? Provide at least two strategies.

Error handling helps an agent continue operating even when problems occur. One strategy is using try-except blocks to catch exceptions and return meaningful error messages. Another strategy is implementing retry mechanisms that automatically repeat failed operations. Logging errors is also useful for debugging and monitoring. These techniques improve reliability and help maintain a better user experience.

9. What is trajectory evaluation in agent systems? Why is it important beyond just checking final output?

Trajectory evaluation examines the entire sequence of actions taken by an agent to solve a task. It focuses on the decisions, tool calls, and intermediate steps used during execution. This helps identify mistakes that may not be visible in the final answer. It is useful for debugging, optimization, and improving agent performance. By evaluating the full trajectory, developers gain a deeper understanding of agent behavior.

10. Define task completion rate and cost metrics. How would you measure and optimize them in a real-world system?

Task completion rate measures the percentage of tasks successfully completed by an agent. Cost metrics represent the resources consumed, such as API calls, processing time, or monetary expenses. These metrics can be measured by tracking agent performance over multiple tasks. To optimize them, developers can improve routing accuracy, reduce unnecessary tool calls, and use efficient algorithms. Balancing high completion rates with low costs is important for real-world deployment.